

D'Shanti Williams & Alyssa Chiquito

Professor Sean Sanders

IT 359 Section 001

May 7, 2026

Vulnerability Scanner

Introduction:

Our vulnerability scanner is a tool that can be used on any website to identify security weaknesses. This tool can be used for defensive or offensive strategies depending on the user's intentions and will give insight into present weaknesses there are within the site. The scanner works by undetectably going through the target website and running a series of checks against known vulnerabilities, returning a detailed report of its findings. Our tool has a straightforward design, requiring only a website URL, making it accessible to both beginner and experienced security professionals. The goal of this project was to use the tool offensively, identify vulnerabilities, and receive the report.

Technical Implementation:

The technical implementation of the vulnerability scanner had several aspects required to create a working tool. We used the Python language to use different libraries when creating functions. This language was the most simplistic and easily learned when having issues completing the code.

Libraries: Collaboratively, it was decided to add several essential libraries when building our web application vulnerability scanning tool. The libraries are as listed: argparse, json, os, re, dataclass, asdict, datetime, urljoin, urlparse, and requests. Argparse was used to create a user-friendly command line interface that accepts and parses through the given URL whereas os is utilized to detect the vulnerabilities within the given target. The re

library was used to search, match, and extract text from scan results. The dataclass and asdict libraries are imported from the dataclasses module where they work together to make the reporting of the vulnerabilities easier to section. Additionally, urljoin and urlparse are coupled up from the urllib.parse module to normalize and resolve the give URL. The request library was used fetch and display security information, datetime was used for timestamps of the logs, and json was utilized for the report output.

Functions: Our code consists of 15 functions in creating an effective scanner. Many of these functions are checking for vulnerabilities such as cookies, common paths, headers, etc. whereas others are methods, utilized within the vulnerability functions. These methods help the functions run smoothly and shorten it to make it much more efficient.

Techniques: There were not too many techniques implemented when building our code. If statements, occasional for loops, exceptions, and at times methods were utilized. The if statements were primarily used in checking for the vulnerabilities and if data matched. For loops were initialized when parsing through required headers and we paired this technique with an if statement to check each item parsed. The exceptions were used to catch any errors that occurred when a function was called. As for methods, there were plenty used, all with different functionalities, but made our code much more concise.

Justification and Analysis:

The development of the Vulnerability Scanner was strategically based on technical components, learning experiences, and practice with real-world cybersecurity applications. The most significant portion of implementing this project was to integrate vulnerability detection with exploitation. This provides consequential documentation of the vulnerability allowing the scanner to detect anomalies, determine potential

weaknesses within the system, relay error messages, and observe the activity of the system without directly interfering with its material. The Vulnerability Scanner + Reporter assists in reducing risks and imitating the structure of a professional penetration testing environment.

For the project, we wanted to integrate into a vulnerable environment that provided multiple exploits that could be identified by our scanner. The OWASP Juice Shop application provided an insecure web application, in which we were able to test and experiment safely, identifying common web application vulnerabilities. We were able to perform authentication testing and observe the responses after testing predefined credentials within the environment. This displayed the importance of being able to seek systematic weaknesses within a system using security testing before engaging in active exploitation. For example, when reporting a vulnerability such as a weak password, reporting would involve performing a minimal analysis and credential audit against the poorly configured credentials within the local host. Ultimately, this exemplifies security testing and vulnerability identification.

The reporting component of the project allowed us to receive valuable insights into the integrity of the target machine. For example, misconfigured cookies suggested that there were risks within session management, and missing security headers suggested that there were unacceptable shortcomings in the defense-in-depth mechanisms. We were able to highlight potential for compromising attacks, through weaknesses within error handling and input validation, which are crucial to note as they would be primary areas of interest for attackers. Every vulnerability identification enhances our understanding of how weak security practices and misconfigurations can lead to substantial security threats and risks. It also allowed us to construct comprehensive remediation recommendations in attempts to patch and rectify the vulnerabilities. Being able to explain how to address

vulnerabilities helps us connect vulnerability identification with risk mitigation, which is an important part of the cybersecurity profession.

Conclusion:

The Vulnerability Scanner + Reporter successfully presents itself as a custom-built Python tool used to execute web application security assessments and provide report generation for easy technical analysis documentation. By integrating reconnaissance, vulnerability testing, authentication analysis, and reporting, the tool properly assists in assessing needs within the security-frail atmosphere.

The tool has a combination of offensive and defensive strategies, security analysis, and integration with web applications. It acts as a foundational piece of risk assessment and deepens the understanding of vulnerability identification.